NEXT ENGINEER · _SHEERAT_

Essentials of Data Structure

ACS201

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** ALL | **Semester:** 2 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 1st year, 2026-27 onwards

UNIT 1 Introduction and Arrays

UNIT 2 Linked Lists

UNIT 3 Stacks and Queues

UNIT 4 Searching and Sorting

UNIT 5 Trees and Graphs (Concepts)

Original revision notes written for the AKTU syllabus. This is a first-year course common to all branches, taught in semester 2, with programs in C and Python. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: operations, algorithms in steps and comparison tables.
3. **Solved traces and programs**: the dry runs examiners ask for (address calculation, linked list code, infix to postfix, postfix evaluation, sorting passes, binary search, BST traversals, BFS and DFS), each checked by running it.
4. **Key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (classification, Big-O, array operations, address formula, matrices) and Unit 2 (singly linked list insert, delete and traverse; doubly and circular lists).
- **Afternoon (3 hours):** Unit 3 (stack and queue with arrays, parenthesis checking, infix to postfix, postfix evaluation) and Unit 4 (linear and binary search; bubble, selection, insertion, quick and merge sort traces).
- **Evening (2 hours):** Unit 5 (binary tree terms, BST insert and search, traversals, graph adjacency list, BFS and DFS), then the "Last-hour checklist".
- **In the exam:** draw boxes and arrows for linked lists, show the stack contents after every symbol, write the array after every sorting pass, and draw the tree before writing traversals.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit. Internal assessment carries another 30 marks.

UNIT 1 Introduction and Arrays

✍️ 2-mark definitions

- **Data structure:** a way of organising data in memory so that it can be used efficiently (array, linked list, stack, queue, tree, graph).
- **Primitive and non-primitive:** primitive types are built into the language (int, char, float); non-primitive ones are built from them (arrays, lists, trees).
- **Linear and non-linear:** in a linear structure, elements form a sequence (array, linked list, stack, queue); in a non-linear structure, an element can connect to many others (tree, graph).
- **Static and dynamic:** static structures have a fixed size decided at compile time (array); dynamic ones grow and shrink at run time (linked list).
- **Abstract data type (ADT):** a description of data and its operations without saying how they are implemented (for example, a stack with push, pop and peek).
- **Algorithm:** a finite set of clear steps that solves a problem and stops.
- **Time complexity:** how the running time grows with input size n. **Space complexity:** how the memory used grows with n.
- **Big-O notation:** an upper bound on growth rate, ignoring constants: O(1), O(log n), O(n), O(n log n), $O(n^2)$.
- **Array:** a fixed-size collection of same-type elements in contiguous memory, accessed by index in O(1).

💡 Core concepts

Classification:

Type	Examples
Primitive	int, char, float, pointer
Linear, static	Array
Linear, dynamic	Linked list, stack, queue (when built with links)
Non-linear	Tree, graph

Growth rates (slowest to fastest): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$. For n = 1,000: log n ≈ 10, n log n ≈ 10,000, n^2 = 1,000,000.

Array operations and their cost:

Operation	Cost	Reason
Access a[i]	O(1)	Address is computed directly
Traverse	O(n)	Visit every element
Insert at position k	O(n)	Shift elements right

Operation	Cost	Reason
Delete at position k	O(n)	Shift elements left
Linear search	O(n)	Check one by one
Binary search (sorted)	O(log n)	Halve the range each step

Address calculation:

- **1D array:** Address(a[i]) = Base + i × w (w = size of one element, index from 0).
- **2D array, row-major (C):** Address(a[i][j]) = Base + (i × N + j) × w, where N = number of columns.
- **2D array, column-major (Fortran):** Address(a[i][j]) = Base + (j × M + i) × w, where M = number of rows.

Insertion in an array (at position pos):

1. If the array is full, report overflow.
2. Shift elements from the last index down to pos one place to the right.
3. Put the new element at pos and increase the size.

🧮 Solved examples and programs

Example 1 (address). int a[10][4] (4 bytes each) starts at address 1000, row-major. Find a[2][3].

- Address = 1000 + (2 × 4 + 3) × 4 = **1044**.

Matrix addition and transpose (C):

```
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++) {
        S[i][j] = A[i][j] + B[i][j];
        T[j][i] = A[i][j];
    }
```

Delete the element at position pos (C):

```
for (i = pos; i < n - 1; i++) a[i] = a[i + 1];
n--;
```

Key points to remember

- Linear vs non-linear; static vs dynamic; ADT means operations, not implementation.
- Big-O order: $1 < \log n < n < n \log n < n^2 < 2^n$.
- Array access O(1); insert and delete O(n).
- Row-major: Base + (iN + j)w; column-major: Base + (jM + i)w.

⭐ Important questions

1. What is a data structure? Classify data structures with examples.

2. What is time and space complexity? Explain Big-O notation with examples of common growth rates.
3. Explain insertion, deletion and traversal in an array with algorithms.
4. Derive address formulas for 1D and 2D arrays (row-major and column-major) and solve a numerical.
5. Write programs for matrix addition and transpose.

NEXT ENGINEER
sheerat

UNIT 2 Linked Lists

✍️ 2-mark definitions

- **Linked list:** a linear collection of **nodes**, where each node holds data and a pointer (link) to the next node. The first node is reached through a **head** pointer; the last node's link is NULL.
- **Singly linked list (SLL):** each node has one link, to the next node. Traversal is in one direction.
- **Doubly linked list (DLL):** each node has two links, prev and next, so it can be traversed both ways.
- **Circular linked list (CLL):** the last node points back to the first, so there is no NULL at the end.
- **Overflow and underflow:** overflow is when no memory is free for a new node (malloc returns NULL); underflow is deleting from an empty list.
- **Self-referential structure:** a structure that contains a pointer to its own type, used to build nodes.

💡 Core concepts

Array vs linked list:

Feature	Array	Linked list
Memory	Contiguous, fixed size	Scattered, grows at run time
Access	Random, O(1)	Sequential, O(n)
Insert or delete at front	O(n) (shifting)	O(1)
Extra memory	None	One pointer per node (two in DLL)
Cache use	Good	Poor

SLL operations (steps):

- **Insert at beginning:** create node; node→next = head; head = node.
- **Insert at end:** create node with next = NULL; go to the last node; last→next = node.
- **Insert after a given node p:** node→next = p→next; p→next = node.
- **Delete first:** temp = head; head = head→next; free(temp).
- **Delete a key:** keep a prev pointer; when found, prev→next = p→next; free(p).
- **Traverse:** p = head; while p != NULL: visit p→data; p = p→next.

DLL and CLL (concept):

- DLL insert after p: new→next = p→next; new→prev = p; if p→next exists, p→next→prev = new; p→next = new.
- DLL can delete a node in O(1) when given a pointer to it, and can go backwards (browser back and forward, undo and redo).
- CLL is useful for round-robin CPU scheduling and playlists that loop.

Programs

SLL in C:

```c
struct node { int data; struct node *next; };

struct node* insertBegin(struct node *head, int x) {
    struct node *t = malloc(sizeof(struct node));
    t->data = x;
    t->next = head;
    return t;                          /* new head */
}

struct node* deleteKey(struct node *head, int key) {
    struct node *p = head, *prev = NULL;
    while (p != NULL && p->data != key) { prev = p; p = p->next; }
    if (p == NULL) return head;        /* not found */
    if (prev == NULL) head = p->next;  /* deleting the first node */
    else prev->next = p->next;
    free(p);
    return head;
}

void display(struct node *p) {
    while (p != NULL) { printf("%d -> ", p->data); p = p->next; }
    printf("NULL\n");
}
```

SLL in Python:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def insert_end(head, x):
    node = Node(x)
    if head is None:
        return node
    p = head
    while p.next:
        p = p.next
    p.next = node
    return head
```

Trace. Start empty; insertBegin 10, 20, 30, then deleteKey 20. The list is 30 → 20 → 10 → NULL, then **30 → 10 → NULL**.

Key points to remember

- Node = data + link; head points to the first node; last link is NULL (except in CLL).
- Insert at the front is O(1); search is O(n).
- Always check for NULL (empty list, key not found) and free deleted nodes.

- DLL has prev and next; CLL last node points to the first.

⭐ Important questions

1. Compare arrays and linked lists. What are the advantages of a linked list?
2. Write algorithms or programs to insert a node at the beginning, end and a given position of a singly linked list.
3. Write a program to delete a node with a given key and to display a singly linked list.
4. Explain doubly linked lists with insertion and deletion (diagrams).
5. What is a circular linked list? Give its applications.

NEXT ENGINEER
sheerat

UNIT 3 Stacks and Queues

✍️ 2-mark definitions

- **Stack:** a linear structure where insertion (push) and deletion (pop) happen at one end, the **top**. It follows **LIFO** (last in, first out).
- **Queue:** a linear structure where insertion happens at the **rear** and deletion at the **front**. It follows **FIFO** (first in, first out).
- **Stack overflow and underflow:** pushing onto a full stack (top = MAX − 1) and popping from an empty stack (top = −1).
- **Call stack:** the stack the system uses for function calls; each call pushes an activation record (parameters, local variables, return address) and returning pops it. Too much recursion gives a stack overflow.
- **Polish notation:** prefix (operator before operands, + A B); **reverse Polish:** postfix (A B +). Neither needs brackets.
- **Circular queue:** a queue in which rear wraps round to index 0, so freed space at the front is reused.
- **Priority queue:** each element has a priority; the highest priority is removed first.
- **Deque:** a double-ended queue allowing insert and delete at both ends.

💡 Core concepts

Stack with an array (top starts at −1):

- **push(x):** if top == MAX − 1, overflow; else top++, s[top] = x.
- **pop():** if top == −1, underflow; else return s[top−−].
- **peek():** return s[top].

Queue with an array (front = rear = −1):

- **enqueue(x):** if rear == MAX − 1, overflow; if empty, front = 0; rear++; q[rear] = x.
- **dequeue():** if front == −1 or front > rear, underflow; else return q[front++].
- In a simple array queue, space at the front is wasted after dequeues; a **circular queue** uses rear = (rear + 1) % MAX to fix this.

Stack vs queue:

Feature	Stack	Queue
Order	LIFO	FIFO
Ends used	One (top)	Two (front and rear)
Operations	push, pop, peek	enqueue, dequeue
Uses	Undo, function calls, recursion, expression conversion, parenthesis matching, browser back	CPU and printer scheduling, BFS, buffers, call centres

Parenthesis matching: scan left to right; push every opening bracket; on a closing bracket, pop and check that it matches. The expression is balanced if no mismatch occurs and the stack is empty at the end.

Infix to postfix (algorithm):

1. Operand: write it to the output.
2. '(': push.
3. ')': pop to the output until '(' is found; discard both brackets.
4. Operator: pop operators of higher or equal precedence (except for right-associative ^, pop only higher) to the output, then push the new one.
5. At the end, pop everything left. - Precedence: ^ highest, then * and /, then + and −.

Postfix evaluation: scan left to right; push operands; on an operator, pop two values (the first popped is the right operand), apply it, push the result. The final value on the stack is the answer.

Solved traces

Trace 1 (infix to postfix). A + B * C − D / E.

Symbol	Stack	Output
A		A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
−	−	A B C * +
D	−	A B C * + D
/	− /	A B C * + D
E	− /	A B C * + D E
end		A B C * + D E / −

Result: `ABC*+DE/-`. Others: (A + B) * (C − D) → `AB+CD-*`; A + B * (C ^ D − E) → `ABCD^E-*+`.

Trace 2 (postfix evaluation). 5 3 + 8 2 − *

- Push 5, push 3; '+' gives 8. Push 8, push 2; '−' gives 6. '*' gives 8 × 6 = **48**.

Trace 3 (balanced brackets). "{[()]}" is balanced; "([)]" is not, because ')' meets '[' on top.

Key points to remember

- Stack LIFO (top = −1 empty, MAX − 1 full); queue FIFO (front, rear).
- Circular queue: rear = (rear + 1) % MAX.
- Infix to postfix: operands out, operators by precedence, brackets on the stack.
- Postfix evaluation: second popped value is the left operand.

⭐ Important questions

1. Explain the stack ADT. Write push, pop and peek using an array with overflow and underflow checks.
2. Explain queue operations with an array. What is the drawback of a linear queue and how does a circular queue solve it?
3. Convert an infix expression to postfix using a stack (show the table) and evaluate a postfix expression.
4. Write a program to check balanced parentheses using a stack.
5. Explain applications of stacks (call stack, recursion) and queues (scheduling).

NEXT ENGINEER
sheerat

UNIT 4 Searching and Sorting

✍️ 2-mark definitions

- **Searching:** finding whether an element is present and where.
- **Linear search:** checks each element in turn. Works on unsorted data; O(n).
- **Binary search:** works only on sorted data; compares with the middle element and halves the range each time; O(log n).
- **Sorting:** arranging elements in ascending or descending order.
- **Stable sort:** keeps equal elements in their original relative order (bubble, insertion, merge are stable; selection and quick are not).
- **In-place sort:** needs only O(1) extra memory (bubble, selection, insertion, quick); merge sort needs O(n) extra.
- **Divide and conquer:** split the problem, solve the parts recursively and combine (quick sort, merge sort, binary search).

💡 Core concepts

Sorting methods:

- **Bubble sort:** compare neighbours and swap if out of order; after each pass the largest remaining element "bubbles" to the end. Stop early if a pass makes no swap.
- **Selection sort:** find the minimum of the unsorted part and swap it to the front.
- **Insertion sort:** take the next element and insert it into the correct place in the sorted left part (like arranging playing cards).
- **Quick sort:** choose a pivot, partition so smaller elements are on the left and larger on the right, then sort both sides recursively.
- **Merge sort:** split the array into halves until single elements remain, then merge sorted halves.

Comparison:

Algorithm	Best	Average	Worst	Extra space	Stable
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

Python sorting: `sorted(lst)` returns a new sorted list; `lst.sort()` sorts in place. Both accept `reverse=True` and `key=` (for example `sorted(words, key=len)`). Python uses Timsort, a stable O(n log n) mix of merge and insertion sort.

Solved traces

Binary search for 31 in [3, 8, 12, 17, 23, 31, 42, 56]:

- low 0, high 7, mid 3 → 17 < 31, so low = 4.
- low 4, high 7, mid 5 → 31 found at **index 5** in 2 comparisons.

Bubble sort [5, 1, 4, 2, 8]:

- Pass 1: [1, 4, 2, 5, 8]
- Pass 2: [1, 2, 4, 5, 8]
- Pass 3: no swap, so stop (already sorted).

Selection sort [29, 10, 14, 37, 13]:

- Pass 1: [10, 29, 14, 37, 13]
- Pass 2: [10, 13, 14, 37, 29]
- Pass 3: [10, 13, 14, 37, 29] (14 already in place)
- Pass 4: [10, 13, 14, 29, 37]

Insertion sort [12, 11, 13, 5, 6]:

- Insert 11: [11, 12, 13, 5, 6]
- Insert 13: [11, 12, 13, 5, 6]
- Insert 5: [5, 11, 12, 13, 6]
- Insert 6: [5, 6, 11, 12, 13]

Quick sort partition (last element as pivot) [10, 80, 30, 90, 40, 50, 70]: pivot 70 → [10, 30, 40, 50, **70**, 90, 80]. The pivot is at index 4; sort [10, 30, 40, 50] and [90, 80] recursively.

Merge sort [38, 27, 43, 3, 9, 82, 10]:

- Split: [38, 27, 43, 3] and [9, 82, 10] → [38, 27] [43, 3] [9, 82] [10] → single elements.
- Merge: [27, 38] [3, 43] [9, 82] [10] → [3, 27, 38, 43] and [9, 10, 82] → **[3, 9, 10, 27, 38, 43, 82]**.

Binary search in C:

```c
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;      /* not found */
}
```

Key points to remember

- Binary search needs sorted data; O(log n).
- Bubble, insertion: best O(n); selection always O(n^2).
- Quick sort worst O(n^2) (already sorted with bad pivot); merge sort always O(n log n) but O(n) space.
- Stable: bubble, insertion, merge. Python sort is stable Timsort.

⭐ Important questions

1. Compare linear and binary search; write the binary search algorithm and trace it.
2. Explain bubble, selection and insertion sort with a pass-by-pass trace and complexity.
3. Explain quick sort with the partition procedure and its best and worst case.
4. Explain merge sort with an example; why is it O(n log n)?
5. Compare all sorting algorithms (table). How do Python's sort() and sorted() differ?

NEXT ENGINEER
sheerat

UNIT 5 Trees and Graphs (Concepts)

✍️ 2-mark definitions

- **Tree:** a non-linear hierarchical structure of nodes connected by edges, with one **root** and no cycles. A tree with n nodes has n − 1 edges.
- **Terms: parent, child, sibling**; **leaf** (no children); **internal node**; **degree** (number of children); **level** (root at 0); **height** (longest path from root to a leaf, in edges); **subtree**.
- **Binary tree:** each node has at most two children (left and right). Maximum nodes at level l = 2^l; maximum nodes in a tree of height h = 2^(h+1) − 1.
- **Full (strict) binary tree:** every node has 0 or 2 children. **Complete binary tree:** all levels full except possibly the last, which is filled from the left.
- **Binary search tree (BST):** a binary tree where every left subtree key is smaller than the node and every right subtree key is larger. An inorder traversal gives sorted order.
- **Graph:** G = (V, E), a set of vertices and edges. It can be **directed** or **undirected**, **weighted** or unweighted, and may have cycles.
- **Degree of a vertex:** number of edges at it (in-degree and out-degree for directed graphs).
- **Adjacency list:** for each vertex, a list of its neighbours; memory O(V + E). **Adjacency matrix:** a V × V table of 0s and 1s; memory $O(V^2)$.

💡 Core concepts

Traversals of a binary tree:

- **Inorder (LNR):** left subtree, node, right subtree.
- **Preorder (NLR):** node, left, right (used to copy a tree).
- **Postorder (LRN):** left, right, node (used to delete a tree).

BST operations:

- **Search:** start at the root; go left if the key is smaller, right if larger; stop when found or NULL. O(h): O(log n) when balanced, O(n) when skewed.
- **Insert:** search for the key; insert the new node where the search ends at NULL.

Tree vs graph:

Feature	Tree	Graph
Root	One root	No root
Cycles	None	Allowed
Edges	n − 1	Any number
Path between two nodes	Exactly one	Zero, one or many
Example	File system, organisation chart	Road map, social network

BFS and DFS:

- **BFS (breadth-first):** uses a **queue**; visits all neighbours of a vertex before going deeper (level by level). Finds the shortest path in an unweighted graph.
- **DFS (depth-first):** uses a **stack** (or recursion); goes as deep as possible before backtracking. Used for cycle detection, topological sorting and maze solving.
- Both take O(V + E) with an adjacency list.

Solved traces

BST. Insert 50, 30, 70, 20, 40, 60, 80 in order. Root 50; 30 and 70 are its children; 20 and 40 under 30; 60 and 80 under 70.

- Inorder: **20 30 40 50 60 70 80** (sorted).
- Preorder: **50 30 20 40 70 60 80**.
- Postorder: **20 40 30 60 80 70 50**.
- Search 60: 50 → right to 70 → left to 60, found in 3 comparisons.

Graph. Adjacency list: A: B, C; B: A, D, E; C: A, F; D: B; E: B, F; F: C, E. Start at A, neighbours in alphabetical order.

- BFS (queue): **A B C D E F**.
- DFS (recursive): **A B D E F C**.

BST insert in Python:

```
class Node:
    def __init__(self, key):
        self.key, self.left, self.right = key, None, None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def inorder(root):
    if root:
        inorder(root.left); print(root.key, end=" "); inorder(root.right)
```

Key points to remember

- Tree: n nodes, n − 1 edges, no cycles; height h has at most 2^(h+1) − 1 nodes.
- Inorder LNR (sorted for BST), preorder NLR, postorder LRN.
- BST search O(log n) balanced, O(n) skewed.
- BFS uses a queue (level order); DFS uses a stack or recursion.
- Adjacency list O(V + E) memory; matrix O(V^2).

⭐ Important questions

1. Define tree terms (root, leaf, degree, level, height) and the types of binary trees with diagrams.
2. Explain inorder, preorder and postorder traversals; find all three for a given tree.
3. What is a BST? Construct a BST from a sequence, and explain search and insert.
4. Represent a graph using an adjacency list and an adjacency matrix. Compare them.
5. Explain BFS and DFS with an example trace. Compare them.

Last-hour checklist

☐ Classification table; Big-O order; array operation costs; address 1044 (row-major formula); matrix programs.

☐ Array vs linked list table; SLL insert and delete code; trace 30 → 10 → NULL; DLL prev and next; CLL uses.

☐ Stack (top −1 to MAX − 1) and queue code; circular queue; `ABC*+DE/-`, `AB+CD-*`, `ABCD^E-*+`; postfix 48; bracket matching.

☐ Binary search index 5; bubble, selection, insertion passes; quick sort pivot 70 at index 4; merge sort result; complexity table; sort() vs sorted().

☐ Tree terms and formulas; BST traversals (20...80, 50 30 20 40 70 60 80, 20 40 30 60 80 70 50); BFS A B C D E F; DFS A B D E F C; list vs matrix.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy. The third book is also published free online by its authors.

#	Title	Author(s)	Publisher	Where to get it legally
1	Data Structures Using C	Reema Thareja	Oxford University Press	WorldCat
2	Data Structures and Algorithms in Python	Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser	Wiley	WorldCat
3	Problem Solving with Algorithms and Data Structures using Python	Bradley N. Miller, David L. Ranum	Franklin, Beedle / Runestone Academy	Free online edition

Syllabus source: AKTU B.Tech 1st year CSE stream course structure and syllabus, 2026-27 (official PDF)